

PROGRAMACION WEB

Bienvenidos!!!

Julio Monetti
julio.monetti@itu.uncu.edu.ar



Javascript. Arrays

```
let frutas = ["Manzana", "Banana", "Pera"];  
frutas.push("Mango");  
console.log(frutas);  
console.log(frutas[1]);
```

Javascript. Arrays

Diferentes formas de creación:

// Declaración simple

```
let numeros = [1, 2, 3, 4, 5];
```

// Array con longitud fija (valores vacíos)

```
let vacios = new Array(5); // [ <5 ítems vacíos> ]
```

// Array lleno con un valor

```
let relleno = Array(5).fill(0); // [0,0,0,0,0]
```

// Array a partir de otro objeto iterable

```
let texto = "Hola";
```

```
let letras = Array.from(texto); // ["H", "o", "l", "a"]
```

// Array generado dinámicamente

```
let cuadrados = Array.from({length: 5}, (_, i) => i * i); // [0,1,4,9,16]
```

Javascript. Arrays

Búsqueda

```
let arr = [10, 20, 30, 40, 50];
```

```
console.log( arr.includes(20) ); // true
```

```
console.log( arr.indexOf(30) ); // 2
```

```
console.log( arr.find(x => x > 25) ); // 30
```

```
console.log( arr.findIndex(x => x > 25) ); // 2
```

Javascript. Arrays

Transformaciones con map, filter y reduce.

```
let nums = [1, 2, 3, 4, 5];
```

```
// Map: transformar
```

```
let dobles = nums.map(n => n * 2); // [2,4,6,8,10]
```

```
// Filter: filtrar
```

```
let pares = nums.filter(n => n % 2 === 0); // [2,4]
```

```
// Reduce: acumular
```

```
let suma = nums.reduce((acc, n) => acc + n, 0); // 15
```

```
// Combinados
```

```
let resultado = nums
```

```
  .filter(n => n % 2 !== 0) // impares [1,3,5]
```

```
  .map(n => n ** 2)        // cuadrados [1,9,25]
```

```
  .reduce((a, b) => a + b); // 35
```

Javascript. Arrays

Práctica a2

Una tienda tiene la siguiente lista de productos:

```
const productos = [  
  { nombre: "Notebook", precio: 1200, stock: 5 },  
  { nombre: "Mouse", precio: 25, stock: 0 },  
  { nombre: "Teclado", precio: 45, stock: 10 },  
  { nombre: "Monitor", precio: 350, stock: 3 },  
  { nombre: "Webcam", precio: 80, stock: 0 },  
  { nombre: "Auriculares", precio: 60, stock: 8 }  
];
```

Desarrollar un programa en JavaScript que:

Utilice `filter()` para obtener solamente los productos que tengan stock disponible (`stock > 0`).

Sobre el resultado anterior, utilice `map()` para generar un nuevo array que contenga únicamente el nombre y el precio de cada producto.

Mostrar el resultado por consola.

Javascript. Arrays

Ordenar y agrupar

Javascript. Arrays

Combinar arreglos

```
let arr1 = [1, 2];
```

```
let arr2 = [3, 4];
```

```
let combinado = [...arr1, ...arr2]; // [1,2,3,4]
```

// Concatenar

```
let arr3 = arr1.concat(arr2); // [1,2,3,4]
```

Javascript. Arrays

Aplanar arreglos

// Aplanar

```
let anidado = [1, [2, 3], [4, [5, 6]]];
```

```
console.log(anidado.flat(2)); // [1,2,3,4,5,6]. El parámetro (2) representa niveles de aplanado
```

// Transformar y aplanar a la vez

```
let frases = ["hola mundo", "adios sol"];
```

```
let palabras = frases.flatMap(f => f.split(" ")); // ["hola","mundo","adios","sol"]
```

Javascript. Arrays

Desestructuración

```
let [primero, segundo, ...resto] = [10, 20, 30, 40];  
console.log(primero); // 10  
console.log(resto); // [30,40]
```

// Intercambiar valores

```
let a = 1, b = 2;  
[a, b] = [b, a];  
console.log(a, b); // 2, 1
```

Javascript. Arrays

Set y Map con Arreglos

// Eliminar duplicados

```
let repetidos = [1,2,2,3,3,4];
```

```
let unicos = [...new Set(repetidos)];    // [1,2,3,4]
```

Set es una colección de valores únicos

// Contar frecuencia

```
let palabras2 = ["sol", "luna", "sol", "estrella", "luna"];
```

```
let conteo = palabras2.reduce((map, p) => {
```

```
    map[p] = (map[p] || 0) + 1;
```

```
    return map;
```

```
}, {});    //empieza con un arreglo vacío
```

```
console.log(conteo); // { sol:2, luna:2, estrella:1 }
```

Javascript. Arrays

Métodos inmutables y de transformación

Para evitar la modificación accidental del array original, JavaScript moderno ofrece alternativas que devuelven nuevas copias:

toReversed(): Invierte el orden de los elementos.

toSorted(compareFn): Ordena los elementos devolviendo un nuevo arreglo.

toSpliced(start, deleteCount, ...items): Reemplaza y elimina elementos sin modificar el array base. Es la versión inmutable de splice(). Mientras que splice() modifica el arreglo original, toSpliced() devuelve un nuevo arreglo

with(index, value): Modifica un valor en un índice específico de forma inmutable

Javascript. Arrays

Métodos inmutables y de transformación

toReversed():

```
const numeros = [1, 2, 3, 4, 5];  
const invertido = numeros.toReversed();
```

toSorted(compareFn):

```
const numeros = [8, 3, 1, 9, 5];  
const ordenados = numeros.toSorted((a, b) => a - b);
```

Javascript. Arrays

Métodos inmutables y de transformación

toSpliced(start, deleteCount, ...items):

Eliminar un elemento

```
const colores = ["Rojo", "Verde", "Azul", "Amarillo"];
```

```
const nuevo = colores.toSpliced(1, 1);
```

Insertar un elemento

```
const numeros = [10, 20, 40, 50];
```

```
const nuevo = numeros.toSpliced(2, 0, 30);
```

Reemplazar un elemento

```
const frutas = [ "Manzana", "Pera", "Banana"];
```

```
const nuevo = frutas.toSpliced(1, 1, "Naranja");
```

Reemplazar varios elementos

```
const letras = ["A", "B", "C", "D", "E"];
```

```
const nuevo = letras.toSpliced( 1, 2, "X", "Y", "Z");
```

Javascript. Arrays

Métodos inmutables y de transformación

with(index, value):

Reemplazar un número

```
const numeros = [10, 20, 30, 40];
```

```
const nuevo = numeros.with(2, 99);
```

Reemplazar un objeto

```
const alumnos = [ { nombre: "Ana", nota: 8 }, { nombre: "Juan", nota: 6 }, { nombre: "Pedro", nota: 9 }];
```

```
const nuevosAlumnos = alumnos.with( 1, { nombre: "Juan", nota: 10 });
```

Javascript. Arrays

every

Para reemplazar un for. Itera un arreglo, evalúa una condición, y devuelve un booleano.

```
const numbers = [1,2,3,4];  
todosnumeros = numbers.every( number => typeof number === 'number')
```

Deja de iterar cuando encuentra un elemento que no cumple la condición. Verifiquemos

```
const numbers = [1, "2", 3, 4];  
todosnumeros = numbers.every((number, index) => {  
  console.log(index);  
  return typeof number === "number";  
});  
console.log(todosnumeros);
```

Javascript. Arrays

some

Para reemplazar un for. Itera un arreglo, evalúa una condición que se cumpla para al menos un elemento, y devuelve un booleano.

```
const numbers = [1, "2", "3", "4"];
```

```
algunnumero = numbers.some((number) => typeof number === "number");
```

```
console.log(algunnumero);
```

Javascript. Arrays

Iteradores

```
let numeros3 = [10, 20, 30];
```

```
// for...of
```

```
for (let n of numeros3) console.log(n);
```

```
// entries() → índice y valor
```

```
for (let [i, n] of numeros3.entries()) console.log(i, n);
```

```
// keys() y values()
```

```
console.log([...numeros3.keys()]); // [0,1,2]
```

```
console.log([...numeros3.values()]); // [10,20,30]
```

Práctica a4_2. Arreglos en Javascript

Realice el siguiente ejercicio en Javascript. Use every.

queremos comprobar que todos los empleados:

Tienen nombre.

Tienen un salario mayor a \$500.000.

Pertenecen a un departamento válido.

Si son del departamento "IT", tienen al menos 2 años de experiencia.

```
const empleados = [  
  {  
    nombre: "Ana",  
    departamento: "IT",  
    salario: 850000,  
    experiencia: 4  
  },  
  {  
    nombre: "Luis",  
    departamento: "Ventas",  
    salario: 650000,  
    experiencia: 3  
  },  
  {  
    nombre: "Marta",  
    departamento: "IT",  
    salario: 900000,  
    experiencia: 5  
  },  
  {  
    nombre: "Pedro",  
    departamento: "Marketing",  
    salario: 600000,  
    experiencia: 2  
  }  
];  
const departamentosValidos = [  
  "IT",  
  "Ventas",  
  "Marketing"  
];
```

Práctica a4_3. Arreglos en Javascript

Realice los siguientes ejercicios en javascript

```
const numeros = [3, 6, 9, 12, 15, 18];
```

Genera un nuevo arreglo que contenga los cubos de los números pares.

```
const empleados = [  
  { nombre: "Ana", departamento: "Ventas" },  
  { nombre: "Luis", departamento: "IT" },  
  { nombre: "Marta", departamento: "Ventas" },  
  { nombre: "Pedro", departamento: "IT" },  
  { nombre: "Sofía", departamento: "Marketing" }  
];
```

Agrupar los empleados por departamento en un objeto

Práctica a4_3. Arreglos en Javascript

Resolución

```
let cubosPares = numeros
  .filter(n => n % 2 === 0)
  .map(n => n * n * n);
```

```
let agrupados = empleados.reduce((acc, emp) => {
  // si no existe aún la clave del departamento, la creo como arreglo vacío
  if (!acc[emp.departamento]) {
    acc[emp.departamento] = [];
  }
  // agrego el empleado al depto correspondiente
  acc[emp.departamento].push(emp);
  return acc;
}, {});
```

Práctica a4_4. Arreglos en Javascript

Buscar el número más repetido de un arreglo.

Práctica a4_41. Arreglos en Javascript

```
const estudiantes = [  
  { nombre: "Ana", nota: 8 },  
  { nombre: "Luis", nota: 5 },  
  { nombre: "Carla", nota: 9 },  
  { nombre: "Juan", nota: 6 },  
  { nombre: "Pedro", nota: 4 }  
];
```

- Obtener un nuevo arreglo solo con los nombres de los estudiantes que aprobaron ($\text{nota} \geq 6$).
- Ordenar el arreglo original por nota de mayor a menor.
- Calcular el promedio general de las notas.

Práctica a4_42. Matrices en Javascript

Dada una matriz

Entrada:

```
[ [1, 2, 3],  
  [4, 5, 6],  
  [7, 8, 9] ]
```

Obtener

- La **suma de cada fila**.
- La **suma de cada columna**.
- Salida:
- Suma de filas: [6, 15, 24]
- Suma de columnas: [12, 15, 18]

Práctica a4_43. Matrices en Javascript

Dada una **matriz cuadrada** ($n \times n$), escribir una función que obtenga:

La **diagonal principal**.

La **diagonal secundaria**.

La **suma de ambas diagonales**.

Práctica a4_44. Matrices en Javascript

Aplanar una matriz tridimensional (pasarla a un arreglo), y ordenarlo.

Javascript. Objetos

En JavaScript, un objeto es una colección de pares **clave-valor**.

Se usa para modelar entidades del mundo real (persona, coche, empresa, etc.).

Javascript. Objetos

```
let persona = {  
  nombre: "Carlos",  
  edad: 30,  
  saludar: function() {  
    console.log("Hola, soy " + this.nombre);  
  }  
};
```

Javascript. Objetos

```
const calculadora = {  
  sumar: function(a, b) {  
    return a + b;  
  },  
  restar(a, b) {  
    return a - b;  
  }  
};
```

```
console.log( calculadora.sumar(5, 3) ); // 8  
console.log( calculadora.restar(10, 4) ); // 6
```

Javascript. Objetos

Referencia al objeto

```
const a = { nombre: "Juan" };
```

```
const b = a;
```

```
b.nombre = "Pedro";
```

```
console.log(a.nombre);
```

Javascript. Objetos

Propiedades Calculadas

Podemos crear propiedades cuyo nombre proviene de una variable.

```
const propiedad = "nombre";  
const persona = {  
  [propiedad]: "Juan"  
};  
console.log(persona.nombre);
```

Esto es especialmente útil cuando construimos objetos dinámicamente.

Javascript. Objetos

Propiedades Calculadas

Construcción a partir de datos

```
const datos = [  
  ["nombre", "Ana"],  
  ["edad", 25],  
  ["ciudad", "Mendoza"]  
];  
  
const persona = {};  
  
for (const [propiedad, valor] of datos) {  
  persona[propiedad] = valor;  
}
```

Javascript. Objetos

Agregar propiedades

```
"use strict";
```

```
objeto = {nombre: "Juan"};
```

```
Object.defineProperty(objeto, "id", {  
  value: 100,  
  writable: false  
});
```

```
objeto.id = 90;  
console.log( objeto.id );
```

Javascript. Objetos

Agregar propiedades

```
Object.defineProperty(persona, "dni", {  
  value: 12345678,  
  writable: false,  
  enumerable: true,  
  configurable: false  
});
```

Enumerable: Determina si la propiedad aparece cuando **enumeramos las propiedades del objeto**.

Configurable: Controla si las **características de la propiedad pueden modificarse o si la propiedad puede eliminarse**.

Javascript. Objetos

Agregar y modificar propiedades

Object.**preventExtensions**(persona);

No permite agregar propiedades nuevas.

Object.**seal**(persona);

No permite agregar ni eliminar propiedades.

Object.**freeze**()

No permite modificar las propiedades.

Javascript. Objetos

Luego, podemos verificar con los métodos

`Object.isExtensible()`

`Object.isSealed()`

`Object.isFrozen()`

Javascript. Objetos

Eliminar Propiedades

```
const persona = {  
  nombre: "Juan",  
  edad: 30,  
  ciudad: "Mendoza"  
};
```

```
delete persona.edad;
```

```
console.log(persona);
```

Javascript. Objetos

Crear objetos dinámicamente

```
const persona = {};
```

```
persona.nombre = "Ana";
```

```
persona.edad = 25;
```

```
console.log(persona);
```

Javascript. Objetos

Crear objetos dinámicamente

```
function crearPersona(nombre, edad) {  
  
    return {  
        nombre: nombre,  
        edad: edad  
    };  
}
```

```
const p1 = crearPersona("Juan", 20);  
const p2 = crearPersona("Ana", 25);
```

Javascript. Objetos

Objetos Anidados

```
const empresa = {  
  nombre: "TechCorp",  
  direccion: {  
    calle: "San Martín",  
    numero: 123  
  }  
};
```

```
console.log(empresa.direccion.calle);
```

Javascript. Objetos

Iterar sobre objetos

```
let persona = { nombre: "Ana", edad: 28, ciudad: "Madrid" };
```

```
for (let clave in persona) {  
  console.log(`${clave}: ${persona[clave]}`);  
}
```

Javascript. Objetos

Keys, values y entries

```
let persona = { nombre: "Ana", edad: 28, ciudad: "Madrid" };
```

```
// Obtener claves y valores
```

```
console.log(Object.keys(persona)); // ["nombre", "edad", "ciudad"]
```

```
console.log(Object.values(persona)); // ["Ana", 28, "Madrid"]
```

```
console.log(Object.entries(persona));  
[["nombre", "Ana"], ["edad", 28], ["ciudad", "Madrid"]]
```

Javascript. Objetos

Reconstrucción mediante **fromEntries**

```
const objeto = Object.fromEntries([  
  ["nombre", "Juan"],  
  ["edad", 30]  
]);  
  
console.log( objeto );
```

Javascript. Objetos

Ejercicio: Convertir todas las propiedades a mayúsculas:

```
const persona = {  
  nombre: "Juan",  
  edad: 30,  
  ciudad: "Mendoza"  
};
```

```
const resultado = Object.fromEntries(  
  Object.entries(persona).map(([clave, valor]) => [  
    clave.toUpperCase(),  
    valor  
  ])  
);
```

Javascript. Objetos

Desestructuración

```
const persona = {  
  nombre: "Juan",  
  edad: 30,  
  ciudad: "Mendoza"  
};
```

```
const { nombre, edad } = persona;
```

```
console.log(nombre);  
console.log(edad);
```

Javascript. Objetos

Desestructuración

Cambiar el nombre de un atributo durante la desestructuración

```
const { nombre: nombrePersona } = persona;  
console.log(nombrePersona);
```

Javascript. Objetos

Desestructuración

Valores por defecto

```
const { nombre, profesion = "Desconocida" } = persona;
```

Javascript. Objetos

Desestructuración

En objetos anidados

```
const empleado = {  
  nombre: "Ana",  
  direccion: {  
    ciudad: "Mendoza",  
    pais: "Argentina"  
  }  
};
```

```
const {  
  direccion: { ciudad }  
} = empleado;
```

Javascript. Objetos

Desestructuración

Con el rest operator

```
const empleado = {  
  nombre: "Ana",  
  apellido: "Perez",  
  direccion: {  
    ciudad: "Mendoza",  
    pais: "Argentina"  
  }  
};  
  
const { nombre, ...otros } = empleado;
```

Javascript. Objetos

Desestructuración

Parámetros de funciones mediante desestructuración

```
function mostrarPersona({ nombre, edad }) {  
    console.log(nombre);  
    console.log(edad);  
}
```

```
mostrarPersona(persona);
```

Javascript. Objetos

Spread operator

```
const persona = {  
  id; 1,  
  nombre: "Juan",  
  edad: 30  
};  
  
const datos = {  
  id: 10,  
  ciudad: "Mendoza",  
  profesion: "Programador"  
};  
  
const personaCompleta = {  
  ...persona,  
  ...datos  
};
```

Javascript. Objetos

Spread operator

El problema de las copias superficiales

```
const persona1 = {  
  nombre: "Juan",  
  direccion: {  
    ciudad: "Mendoza"  
  }  
};
```

```
const persona2 = {  
  ...persona1  
};  
  
persona2.direccion.ciudad = "Buenos Aires";  
console.log(persona1.direccion.ciudad);
```

Javascript. Objetos

Spread operator

El problema de las copias superficiales

```
const persona1 = {  
  nombre: "Juan",  
  direccion: {  
    ciudad: "Mendoza"  
  }  
};
```

```
const persona2 = structuredClone(persona1);
```

```
persona2.direccion.ciudad = "Buenos Aires";  
console.log(persona1.direccion.ciudad);
```

Javascript. Objetos

Spread operator

El problema de las copias superficiales

```
const persona1 = {  
  nombre: "Juan",  
  direccion: {  
    ciudad: "Mendoza"  
  }  
};
```

```
const persona2 = JSON.parse( JSON.stringify(persona1) );
```

```
persona2.direccion.ciudad = "Buenos Aires";  
console.log(persona1.direccion.ciudad);
```

Javascript. Objetos

Operador in

```
const persona = {  
  nombre: "Juan",  
  edad: 30  
};  
console.log("nombre" in persona);
```

O bien...

```
console.log(persona.hasOwnProperty("nombre"));
```

Javascript. Objetos

Método hasOwn

```
const persona = {  
  nombre: "Juan",  
  edad: 30,  
  ciudad: "Mendoza"  
};  
  
console.log( "toString" in persona );  
console.log( Object.hasOwn(persona, "toString") );
```

Porque toString no es una propiedad propia de persona: normalmente es una propiedad heredada de Object.prototype

Javascript. Objetos

Map con objetos

```
const productos = [  
  { nombre: "Monitor", precio: 200000 },  
  { nombre: "Teclado", precio: 50000 },  
  { nombre: "Mouse", precio: 30000 }  
];  
  
const nombres = productos.map(producto => producto.nombre);  
  
console.log(nombres);
```

Javascript. Objetos

Filter con objetos

```
const productos = [  
  { nombre: "Monitor", precio: 200000 },  
  { nombre: "Teclado", precio: 50000 },  
  { nombre: "Mouse", precio: 30000 }  
];
```

```
const resultado = productos.filter(  
  producto => producto.precio > 40000  
);
```

```
console.log(resultado);
```

Javascript. Objetos

Reduce con objetos

```
const productos = [  
  { nombre: "Monitor", precio: 200000 },  
  { nombre: "Teclado", precio: 50000 },  
  { nombre: "Mouse", precio: 30000 }  
];  
  
const total = productos.reduce( (suma, producto) => suma + producto.precio, 0 );  
  
console.log(total);
```

Javascript. Objetos

Prototipos

```
const persona = {  
  nombre: "Juan"  
};
```

```
console.log(  
  Object.getPrototypeOf(persona)  
);
```

El **prototipo de un objeto en JavaScript** es otro objeto que JavaScript utiliza como **fuentes de propiedades y métodos heredados**.

Javascript. Objetos

Prototipos

```
const personaBase = {  
  saludar() {  
    console.log("Hola");  
  }  
};  
  
const juan = Object.create(personaBase);  
juan.nombre = "Juan";  
juan.saludar();
```

Cadena de Prototipos

juan



personaBase



Object.prototype



null

Javascript. Objetos

Prototipos

Analizar el funcionamiento

```
const persona = {  
  nombre: "Juan",  
  saludar() {  
    console.log("Hola " + this.nombre);  
  }  
};
```

```
persona.saludar();  
const f = persona.saludar;  
f();
```

Javascript. Objetos

Prototipos

Analizar el funcionamiento

```
const persona = {  
  nombre: "Juan",  
  saludar() {  
    console.log("Hola " + this.nombre);  
  }  
};
```

```
const f = persona.saludar.bind(persona);  
f();
```

Javascript. Objetos

Clases

Cuando necesitamos crear muchos objetos con la misma estructura podemos utilizar una **clase**.

```
class Persona {  
  constructor(nombre, edad) {  
    this.nombre = nombre;  
    this.edad = edad;  
  }  
  saludar() {  
    console.log("Hola, soy " + this.nombre);  
  }  
}
```

```
const p1 = new Persona("Juan", 30);  
const p2 = new Persona("Ana", 25);
```

Javascript. Objetos

Clases. Propiedades Privadas

```
class Persona {  
    #dni;  
  
    constructor(nombre, dni) {  
        this.nombre = nombre;  
        this.#dni = dni;  
    }  
    mostrarDni() {  
        console.log(this.#dni);  
    }  
}
```

Javascript. Objetos

Clases. Getters y Setters

```
class Persona {  
  constructor(nombre) {  
    this.nombre = nombre;  
  }  
  
  get nombreMayuscula() {  
    return this.nombre.toUpperCase();  
  }  
}
```

```
const p = new Persona("Juan");  
console.log(p.nombreMayuscula);
```

Javascript. Objetos

Clases. Métodos estáticos

```
class MathUtil {  
    static sumar(a, b) {  
        return a + b;  
    }  
}  
  
console.log(MathUtil.sumar(10, 20));
```

Javascript. Objetos

Clases. Métodos estáticos

```
class Persona {  
  saludar() {  
    console.log("Hola desde una instancia");  
  }  
  static probar() {  
    console.log("Hola desde un método de clase");  
    this.saludar(); //Error. this representa a la clase Persona, no una instancia de Persona  
  }  
}
```

```
let p1 = new Persona();  
p1.probar(); //error: usar la clase como prefijo
```

Javascript. Objetos

Clases. Métodos estáticos

```
class Persona {  
  saludar() {  
    console.log("Hola desde una instancia");  
  }  
  static probar() {  
    const p = new Persona();  
  
    p.saludar(); // ahora llamo el método de instancia desde una instancia  
  }  
}  
  
Persona.probar();
```

Javascript. Objetos

Clases. Métodos estáticos

Para probar

```
class Persona {  
  saludar() {  
    console.log(this);  
  }  
  static probar() {  
    console.log(this);  
  }  
}  
  
const p = new Persona();  
p.saludar();    // this → p  
Persona.probar(); // this → Persona
```

Javascript. Objetos

Clases. Herencia y Polimorfismo

```

class Persona {
  constructor(nombre) {
    this.nombre = nombre;
  }

  presentarse() {
    return `Soy ${this.nombre}`;
  }
}

```

```

class Empleado extends Persona {
  constructor(nombre, puesto) {
    super(nombre);
    this.puesto = puesto;
  }

  presentarse() {
    return `${super.presentarse()} y soy ${this.puesto}`;
  }
}

```



Javascript. Objetos

Clases. Proxy

```
const persona = {
  nombre: "Juan",
  edad: 30
};

const proxy = new Proxy(persona, {
  get(objeto, propiedad) {
    console.log("Accediendo a:", propiedad);
    return objeto[propiedad];
  },
  set(objeto, propiedad, valor) {
    console.log(`Modificando ${propiedad}: ${valor}`);
    objeto[propiedad] = valor;
    return true;
  }
});

proxy.nombre = "Ana";
console.log(proxy.nombre);
```

proxy.nombre

|



Proxy

|



get()

|



objeto.nombre

Javascript. Objetos

Clases. Proxy

También se pueden evaluar métodos:

`has()`

`deleteProperty()`

`ownKeys()`

`getOwnPropertyDescriptor()`

Javascript. Objetos

Práctica a4_441

- Crea un objeto Libro con propiedades titulo, autor, anio. Agrega un método descripcion() que muestre la información.
- Modela un objeto cuentaBancaria con propiedades saldo, titular. Métodos: depositar(monto) y retirar(monto).
- Crea un objeto Rectangulo con ancho y alto. Métodos: area() y perimetro().

Javascript. Objetos

Práctica a4_441. Continuación.

Dado

```
const inventario = {  
  teclado: { precio: 50000, stock: 10 },  
  mouse: { precio: 25000, stock: 20 },  
  monitor: { precio: 180000, stock: 5 }  
};
```

Realizar funciones que permitan:

Calcular el valor total del inventario.

Encontrar el producto más caro.

Mostrar únicamente productos sin stock.

Incrementar el precio un 15%.

Javascript. Objetos

Práctica a4_441. Continuación.

Escribir una función **deepEqual(obj1,obj2)** que realiza la comparación profunda de dos objetos.

```
const a = {  
  x:10,  
  y:{  
    z:20  
  }  
};  
const b = {  
  x:10,  
  y:{  
    z:20  
  }  
};  
console.log(deepEqual(a,b));
```

devolvería true

Javascript. Objetos

Práctica a4_441. Continuación.

Escribir una función `merge(obj1,obj2)`

que permita fusionar objetos recursivamente.

Javascript. Objetos

Práctica Objetos2.

(Ver en la plataforma)

Javascript. Objetos

Práctica Objetos3.

(Ver en la plataforma)

Javascript. Objetos

Práctica Objetos4. (Instrospección de objetos)

(Ver en la plataforma)